

## Lab Sheet: 3

### Objectives

- To understand the use of COMMIT and ROLLBACK commands in transaction control.
- To use ORDER BY and GROUP BY clauses in SQL queries.
- To understand DCL commands (GRANT, REVOKE) and the use of SAVEPOINT.
- To understand and implement JOIN operations in SQL.

### Instructions

Open your preferred Database Management System (DBMS) tool.

1. Create the required tables: TEACHER, SALES, DEPARTMENTS, and EMPLOYEES.
2. Insert suitable records into each table.
3. Execute the SQL commands for transaction control, privilege control, grouping, sorting, and joins.
4. Observe the output after each operation such as ROLLBACK, COMMIT, SAVEPOINT, GRANT, and REVOKE.

### DDL Questions (Data Definition Language)

1. Create the **TEACHER** table with the following attributes: (Name, DeptNo, Date\_of\_Joining, DeptName, Location, Salary).
2. Create the **SALES** table with the following attributes: (Sales\_No, Salesname, Branch, Salesamount, DOB).

3. Create the **DEPARTMENTS** table with the following attributes:  
(dept\_no, dept\_name, dept\_location).
  4. Create the **EMPLOYEES** table with the following attributes:  
(emp\_id, emp\_name, emp\_salary, dept\_no).
- 

## DML Questions (Data Manipulation Language)

### From **TEACHER** Table

1. Insert five records into the **TEACHER** table.
2. Give an increment of 25% salary to teachers working in the Mathematics department.
3. Give an increment of 15% salary to teachers working in the Commerce department.

### From **SALES** Table

1. Insert five records into the **SALES** table.
2. Calculate the total sales amount in each branch.
3. Calculate the average sales amount in each branch.
4. Display the Salesname and DOB of salesmen born in the month of December with date format like 21-Dec-09.
5. Display the Salesname and DOB of salesmen in alphabetical order of the month.

## From JOIN Operations

1. Display employee details and department details where the department numbers are the same in both tables.
2. Display the employee name and department name using a LEFT OUTER JOIN.
3. Display the employee name and department name using a RIGHT OUTER JOIN.
4. Display the details of employees who draw a salary greater than the average salary.

## TCL Questions (Transaction Control Language)

1. Perform the **ROLLBACK** command on the TEACHER table.
2. Perform the **COMMIT** command on the TEACHER table.
3. Write a query to implement a **SAVEPOINT**.

## DCL Questions (Data Control Language)

1. Write a query to **grant all privileges** on the EMPLOYEES table to DEPARTMENTS.
2. Write a query to **grant selected privileges** on the EMPLOYEES table to DEPARTMENTS.
3. Write a query to **revoke all privileges** on the EMPLOYEES table from DEPARTMENTS.
4. Write a query to **revoke selected privileges** on the EMPLOYEES table from DEPARTMENTS.

# ANSWERS

## DDL Questions (Data Definition Language)

1. Create the TEACHER table with the following attributes: (Name, DeptNo, Date\_of\_Joining, DeptName, Location, Salary).

```
CREATE DATABASE IF NOT EXISTS DBMS_Lab3;
USE DBMS_Lab3;

DROP TABLE IF EXISTS TEACHER;
CREATE TABLE TEACHER (
    Name VARCHAR(50),
    DeptNo INT,
    Date_of_Joining DATE,
    DeptName VARCHAR(50),
    Location VARCHAR(50),
    Salary DECIMAL(10,2)
);
```

2. Create the SALES table with the following attributes: (Sales\_No, Salesname, Branch, Salesamount, DOB).

```
DROP TABLE IF EXISTS SALES;
CREATE TABLE SALES (
    Sales_No INT PRIMARY KEY,
    Salesname VARCHAR(50),
    Branch VARCHAR(50),
    Salesamount DECIMAL(10,2),
    DOB DATE
);
```

3. Create the DEPARTMENTS table with the following attributes: (dept\_no, dept\_name, dept\_location).

```
DROP TABLE IF EXISTS DEPARTMENTS;
CREATE TABLE DEPARTMENTS (
    dept_no INT PRIMARY KEY,
    dept_name VARCHAR(50),
    dept_location VARCHAR(50)
);
```

4. Create the EMPLOYEES table with the following attributes: (emp\_id, emp\_name, emp\_salary, dept\_no).

```
DROP TABLE IF EXISTS EMPLOYEES;
CREATE TABLE EMPLOYEES (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    emp_salary DECIMAL(10,2),
    dept_no INT
);
```

```
dept_no INT  
);
```

## DML – TEACHER Table Operations

### From TEACHER Table

#### 1. Insert five records into the TEACHER table.

```
INSERT INTO TEACHER (Name, DeptNo, Date_of_Joining, DeptName, Location, Salary) VALUES  
( 'Mahkar', 101, '2015-06-10', 'Mathematics', 'Delhi', 50000),  
( 'Vinay', 102, '2016-07-12', 'Commerce', 'Mumbai', 45000),  
( 'Dipesh', 103, '2017-08-15', 'Science', 'Chennai', 40000),  
( 'Mayank', 101, '2018-09-20', 'Mathematics', 'Bangalore', 55000),  
( 'Alex', 102, '2019-10-25', 'Commerce', 'Kolkata', 48000);
```

#### 2. Give an increment of 25% salary to teachers working in the Mathematics department.

```
UPDATE TEACHER  
SET Salary = Salary * 1.25  
WHERE DeptName = 'Mathematics';
```

#### 3. Give an increment of 15% salary to teachers working in the Commerce department.

```
UPDATE TEACHER  
SET Salary = Salary * 1.15  
WHERE DeptName = 'Commerce';  
  
SELECT * FROM TEACHER;
```

### From SALES Table

#### Q1. Insert five records into the SALES table.

```
INSERT INTO SALES (Sales_No, Salesname, Branch, Salesamount, DOB) VALUES  
(1, 'Mahkar', 'Delhi', 15000, '1985-12-21'),  
(2, 'Vinay', 'Mumbai', 20000, '1990-11-15'),  
(3, 'Dipesh', 'Delhi', 18000, '1988-12-05'),  
(4, 'Mayank', 'Chennai', 22000, '1992-07-10'),  
(5, 'Alex', 'Mumbai', 25000, '1987-12-30');
```

#### Q2. Calculate the total sales amount in each branch.

```
SELECT Branch, SUM(Salesamount) AS Total_Sales  
FROM SALES  
GROUP BY Branch;
```

#### Q3. Calculate the average sales amount in each branch.

```
SELECT Branch, AVG(Salesamount) AS Avg_Sales  
FROM SALES  
GROUP BY Branch;
```

#### Q4. Display the Salesname and DOB of salesmen born in December (format: 21-Dec-09).

```
SELECT Salesname, DATE_FORMAT(DOB,'%d-%b-%y') AS DOB
```

```
FROM SALES
WHERE MONTH(DOB) = 12;
```

**Q5. Display the Salesname and DOB of salesmen in alphabetical order of the month.**

```
SELECT Salesname, DATE_FORMAT(DOB,'%d-%b-%y') AS DOB
FROM SALES
ORDER BY MONTH(DOB), DAY(DOB);
```

## From JOIN Operations

**Q1. Display employee details and department details where the department numbers are the same.**

```
SELECT e.emp_id, e.emp_name, e.emp_salary, d.dept_name, d.dept_location
FROM EMPLOYEES e
JOIN DEPARTMENTS d
ON e.dept_no = d.dept_no;
```

**Q2. Display employee name and department name using LEFT OUTER JOIN.**

```
SELECT e.emp_name, d.dept_name
FROM EMPLOYEES e
LEFT JOIN DEPARTMENTS d
ON e.dept_no = d.dept_no;
```

**Q3. Display employee name and department name using RIGHT OUTER JOIN.**

```
SELECT e.emp_name, d.dept_name
FROM EMPLOYEES e
RIGHT JOIN DEPARTMENTS d
ON e.dept_no = d.dept_no;
```

**Q4. Display details of employees who draw a salary greater than the average salary.**

```
SELECT *
FROM EMPLOYEES
WHERE emp_salary > (SELECT AVG(emp_salary) FROM EMPLOYEES);
```

## TCL Questions (Transaction Control Language)

**Q1. Perform the ROLLBACK command on the TEACHER table.**

```
SET autocommit = 0;

INSERT INTO TEACHER VALUES('TestUser', 105, '2023-01-01', 'Arts', 'Pune', 40000);

ROLLBACK;
```

**Q2. Perform the COMMIT command on the TEACHER table.**

```
SET autocommit = 0;

INSERT INTO TEACHER VALUES('TestUser2', 106, '2023-02-01', 'Science', 'Mumbai', 45000);
```

```
COMMIT;
```

**Q3. Write a query to implement a SAVEPOINT.**

```
SET autocommit = 0;
```

```
INSERT INTO TEACHER VALUES('TempUser', 107, '2023-03-01', 'Commerce', 'Delhi', 42000);
```

```
SAVEPOINT SP1;
```

```
ROLLBACK TO SP1;
```

```
COMMIT;
```

### **DCL Questions (Data Control Language)**

**Q1. Grant all privileges on EMPLOYEES table to DEPARTMENTS.**

```
GRANT ALL PRIVILEGES ON DBMS_Lab3.EMPLOYEES TO 'departments_user'@'localhost';
```

**Q2. Grant selected privileges (SELECT, INSERT) on EMPLOYEES table to DEPARTMENTS.**

```
GRANT SELECT, INSERT ON DBMS_Lab3.EMPLOYEES TO 'departments_user'@'localhost';
```

**Q3. Revoke all privileges on EMPLOYEES table from DEPARTMENTS.**

```
REVOKE ALL PRIVILEGES ON DBMS_Lab3.EMPLOYEES FROM 'departments_user'@'localhost';
```

**Q4. Revoke selected privileges (INSERT) on EMPLOYEES table from DEPARTMENTS.**

```
REVOKE INSERT ON DBMS_Lab3.EMPLOYEES FROM 'departments_user'@'localhost';
```